# Track A — Red Team Proposal (One-Week Scope)

**Project:** Assessing an existing public-facing AI chatbot **Track:** Track A — Red Team (test and report on an AI application) **Primary attack type:** Prompt injection (direct; indirect only if explicitly authorized) **Authorization route:** A published bug bounty or vulnerability disclosure program whose scope covers the asset, or written owner permission **Timeline:** One week, subject to authorization **Status:** Revised proposal for mentor review

## Capstone Scope

Following mentor feedback, I will assess **one chatbot already deployed on a company website**, rather than build a RAG agent. The company may be based in South Africa or elsewhere. I will select the target with my mentor and record the company, chatbot, URL, owner contact, and approved test scope before starting.

The project will focus on **prompt injection**, a technique covered in the Week 7 detector and guardrails labs:

1. **Direct prompt injection:** test whether user-supplied instructions can make the chatbot disregard its stated role or other documented behavior.
2. **Indirect prompt injection:** test only if the chatbot owner explicitly authorizes an approved test document or input source. I will not plant instructions in third-party content or try to change a company's knowledge base without explicit permission.

Week 8 covered poisoning model training data and model backdoors. That is related background, but it is not the same as changing a live company's RAG corpus. **Knowledge-base poisoning is out of scope** unless the company separately authorizes a specific test dataset and write path, and my mentor approves the expanded scope.

## 1. Target and Authorization

The target is one existing chatbot on a company website, to be selected with my mentor. The target must sit inside a **published bug bounty or vulnerability disclosure program whose scope explicitly covers the specific chatbot URL**, or have written owner permission directly. A public-facing chatbot with no published programme is not an acceptable target regardless of how convenient it is to reach, and a mentor's approval does not substitute for either of those.

I will look for a programme carrying explicit safe-harbour language — for example HackerOne's Gold Standard Safe Harbor, which is enabled by default on new programmes, or its AI Research Safe Harbor, which covers probing model behaviour, unintended outputs, and safety bypasses. Bugcrowd programmes carrying the disclose.io standard are the equivalent. Safe harbour is what makes the work lawful rather than tolerated, so its absence is disqualifying rather than a complication to work around.

Two limits on that protection, which I need to state rather than assume. Safe harbour covers **only the assets listed in the programme brief**; anything outside the published scope is untested no matter what the safe-harbour wording says. And safe harbour binds the programme owner, not their infrastructure providers, so a chatbot that proxies to a third-party service may sit outside what the owner can authorise. Where the scope page is ambiguous about a specific URL or feature, I will ask the programme contact in writing before sending a single request.

Before any active testing, the authorization record will identify the chatbot and allowed test methods, accounts, dates, request limits, data boundaries, and reporting contact.

If permission is unavailable or the programme does not allow prompt-injection testing, I will not probe that chatbot. I will ask my mentor to approve another authorized target or adjust the project to a non-invasive review of publicly available materials.

## 2. Why This Target and Scope

- **Real deployment:** Results concern a chatbot people can actually use, rather than a locally built demonstration.
- **A testable failure mode:** I will pick a target where injection could plausibly cross a trust boundary — an assistant with tool access, one that reads user-supplied or external content, or one serving multiple accounts — rather than a public FAQ widget with nothing behind it. See Section 4 for why this determines whether the project produces a finding.
- **Bootcamp connection:** Week 7 covered direct and indirect prompt injection, detector limitations, guardrails, and red-team probing. I will use those concepts and report the limits of what I can test.
- **One-week feasibility:** Testing one chatbot and one primary attack type keeps the work achievable without building a RAG system or classifier.
- **Responsible testing:** Testing is limited to the system, methods, and test data explicitly approved by its owner.

## 3. Objectives

1. Establish normal chatbot behavior using a small set of benign baseline questions, repeated enough times to know what ordinary variance looks like on this target.
2. Determine what the chatbot is built to protect — its stated scope, its guardrails, its system prompt, and whether it holds tools or other users' data.
3. Within the written scope, test whether prompt injection produces an effect that **crosses a trust boundary**, rather than merely an unusual response.
4. Test indirect prompt injection only if the owner provides or approves a controlled source for that test.
5. Record the exact method, prompt category, response, and outcome for each executed test, including unsuccessful attempts.
6. Report evidence-based findings, limitations, and practical mitigations. I will not describe an untested weakness as a vulnerability.

## 4. Planned Tests

**The bar for calling something a vulnerability.** Prompt injection that makes a chatbot produce text its operator would not have wanted is a *known limitation of the technique*, not a finding. Every published disclosure programme treats it that way, and reports resting on it come back marked informational. A finding requires the injection to cross a trust boundary and affect something the target was actually built to protect. I will classify every result against these five outcomes:

1. **Data belonging to another user** becomes visible to me.
2. **An action I could not perform directly** is performed on my behalf.
3. **A secret leaves the system prompt** — an API key, internal hostname, credential, or third-party integration detail. The prompt text itself is not a secret and I will not report it as one.
4. **The output reaches a downstream sink** — markup rendered as code, an image or link reference resolved by the server, an unescaped value into another system.
5. **One user's input changes what another user sees.**

Where a result meets none of these, I will record it as an observation about behavior and say plainly that it is not a security finding.

The exact prompts and variants will be agreed with the owner before testing. They will use benign, non-sensitive content and stay within the permitted request volume.

| ID | Test | What it checks | Evidence and success measure |
|---|---|---|---|
| 1.1 | Baseline behavior | How the chatbot responds to ordinary in-scope questions, to clearly out-of-scope benign questions, and how much the response varies when the same question is asked repeatedly | 5–10 repetitions per baseline prompt. This is the variance reference every later result is read against, so it is measured before any payload is sent |
| 1.2 | System prompt and configuration disclosure | What the chatbot reveals about its own instructions, connected tools, data sources, and guardrails | Captured output, checked against outcome 3. A prompt with no keys or internal endpoints is an observation, not a finding |
| 1.3 | Direct injection with demonstrable impact | Whether an instruction can cause the chatbot to act beyond the requesting user's authority, reach another account's data, or resolve a server-side reference | Captured transcript showing the boundary crossed, mapped to outcomes 1, 2, 4 or 5. Anything short of a crossed boundary is recorded as blocked or observed-only |
| 1.4 | Output-handling injection | Whether model output containing markup, links, or image references is rendered or fetched unsafely by the surrounding application | The raw response plus what the page did with it, mapped to outcome 4. This is a rendering flaw in the application, not a model behavior, and I will describe it that way |
| 1.5 | Direct injection variants and evasion | Whether paraphrased, role-framed, translated, or simply obfuscated forms change the result, and whether a guardrail catches them for the right reason | Each variant repeated 5–10 times and compared against 1.1 variance, with the guardrail's behavior recorded if observable. A variant that succeeds once out of ten is reported as unreliable, not as a bypass |
| 1.6 | Indirect injection (conditional) | Whether an instruction in an owner-approved test document or source affects the response when the chatbot processes that source | Run only with the owner's explicitly supplied or approved test source; otherwise marked not run |

I will not try to access other users' data, retrieve credentials or secrets, make purchases or account changes, trigger external actions, send data to an external sink, evade access controls, or disrupt service. Outcomes 1 and 2 above describe what a *successful* injection would look like as evidence; obtaining them is only ever attempted where the programme scope explicitly authorises that class of test, and I will not attempt either if scope is ambiguous. Any test involving real personal data, a third party's data, or a live transaction is out of bounds and will not be run.

## 5. Bootcamp Alignment and Exclusions

- **In scope:** Prompt injection, based on the Week 7 prompt-injection detector and chatbot/guardrails exercises. Variants are test cases for that same technique, not separate claims of newly learned attack methods.

- **Conditional:** Indirect injection, only through an owner-approved test source and within written rules of engagement.
- **Out of scope as independent test classes:** RAG knowledge-base poisoning, training-data poisoning, model backdoors, pixel-level adversarial examples/FGSM, and denial-of-service testing.

A distinction worth stating explicitly, because it decides what the report may claim. **Cross-user data access, authorization bypass and SSRF are out of scope as vulnerability classes I go looking for on their own** — those are conventional application-security tests that need the kind of written access to a system I am not going to have. But they are exactly what a *successful prompt injection* might give me, and demonstrating that chain is the strongest result this project can produce. So they appear in Section 4 only as the **impact** of an injection, and only where the programme scope authorises testing that class. I will not pivot into testing them directly, and I will not report an injection as high severity merely because the underlying class would be serious if directly reachable.

The Week 8 data-poisoning and SHA-256 labs are useful context for understanding data integrity, but they do not authorise changes to a real company's documents, training data, or model, and I will not attempt any. I am also not attempting model or training-data extraction, membership inference, or memorization probing: all of them need a large query budget that the request limits will not permit, and a small sample would produce numbers too weak to report.

My Week 7 detector gets used, but offline. I will run it against my own drafted prompts locally to discard the ones it already blocks confidently, and send only the remainder to the live target. That reduces request volume, keeps testing gentle, and reuses the bootcamp work rather than discarding it.

## 6. Rules of Engagement

- Test **one named chatbot only**, at the URL and during the dates stated in the written authorization.
- Use only approved test accounts and owner-provided non-sensitive data. Do not use real customer or employee information.
- Keep request volume low and manual unless automation is expressly approved. Do not run broad scans or denial-of-service tests.
- Do not attempt to access another user's account, private records, credentials, internal systems, or external services.
- Do not publish prompts or findings publicly. Send the report to the named contact and mentor, and follow the owner's disclosure instructions.
- Stop immediately if the chatbot exposes real personal data, affects a real user or transaction, behaves unexpectedly in a way that could cause harm, or the owner asks testing to stop. Preserve minimal evidence and notify the agreed contact.
- If the owner has not authorized active testing, do not execute the attack tests.

## 7. Method and Evidence

A production chatbot is stochastic, so a single response proves almost nothing. Every result in this project is a distribution, not an observation. For each permitted test, I will:

1. Record the date, chatbot URL, account/test context, authorization boundary, and the baseline variance from 1.1.
2. Run the agreed prompt **5–10 times**, at the agreed rate, within the request limit. Do not expand the test without approval.

3. Save every individual input and response rather than only the most interesting one, with timestamps and a screenshot or other permitted evidence. Redact personal information and secrets.
4. Report the **success rate with its denominator**, and treat anything under a consistent threshold as unreliable rather than as a bypass. The threshold is set from 1.1, not chosen afterwards to make a result look clean.
5. Classify each result as **blocked**, **not blocked**, **inconclusive**, or **not run**, and separately as **finding** or **observation** against the five outcomes in Section 4. Those are two different axes and the report keeps them separate.
6. Stop when the approved volume is used or the owner asks, and report what was not covered.

Keeping every response matters for a specific reason: a selective write-up of only the successes is what makes an inexperienced report untrustworthy, and the unsuccessful prompts are evidence that the testing was real.

I will use the Week 7 lab materials to guide manual test categories. I will not use Garak or other automated probing against a live target unless the owner explicitly approves that tool and its limits — automated scanning is both a load on someone else's production system and a good way to walk outside a narrow scope without noticing.

## 8. Deliverables

1. **Target and authorization record:** company, chatbot URL, owner contact, authorization evidence, and rules of engagement.
2. **Test log:** each planned test marked as executed, blocked, inconclusive, or not run.
3. **Evidence:** approved screenshots and exact prompt/response pairs, with sensitive information redacted.
4. **Comprehensive findings report:** executive summary; scope and methods; tasks executed; chatbot responses; successful and unsuccessful tests; evidence; limitations; risk and practical remediation for each substantiated vulnerability.
5. **Disclosure record:** date and method the report was delivered to the owner and mentor, following the agreed disclosure process.

## 9. One-Week Plan

**Before Day 1 — Select and authorize the target.** Agree on one company chatbot with the mentor. Obtain written owner permission and a bounded rules-of-engagement document. If permission is not in place, do not start active tests.

**Day 1 — Confirm scope and baseline.** Verify the exact URL, account, permitted prompts, request limits, stop conditions, and reporting contact. Capture benign baseline behavior.

**Day 2 — Prepare the test log.** Draft the direct-injection prompts for tests 1.2, 1.3, 1.4 and 1.5, run them past my Week 7 detector offline to drop the ones it already blocks confidently, and turn the survivors into a short, bounded checklist. Confirm the evidence and data-handling process with the mentor.

**Days 3–4 — Execute approved tests.** Run 1.1 baseline repetitions first, then 1.2, 1.3, 1.4 and 1.5 within the approved volume. Test 1.6 only if the owner has explicitly approved and provided the source.

**Day 5 — Verify evidence.** Repeat selected results only within the authorised limits. Mark results that cannot be reproduced as inconclusive, and check that every response set was captured rather than just the interesting ones.

**Days 6–7 — Report and disclose.** Write the findings report, include successful and unsuccessful tests, and deliver it privately to the owner and mentor as agreed.

## 10. Reporting and Success Measures

The report will distinguish three things that are easy to blur: a chatbot response that is merely unexpected, a reproducible failure of the chatbot's own guardrails, and a genuine security finding where a trust boundary was crossed. Only the third is reported as a vulnerability, and each is labelled as such.

The report will include:

- Number of tests planned, executed, blocked, inconclusive, and not run.
- Every approved prompt with its full response set and success rate, with identifying or sensitive material redacted.
- For each result, whether it met one of the five outcomes in Section 4, and the evidence for that claim rather than my assertion that it succeeded.
- The baseline variance, so a reader can judge whether a 3-in-10 result is a bypass or noise.
- Findings with severity rationale, limitations, and recommended mitigations.
- An explicit statement of what I did not test and why — scope exclusions, unavailable request budget, and untestable surfaces.
- Confirmation that testing stayed within the written scope and that results were disclosed privately.

Two results are both acceptable, and I will not inflate either. A **confirmed finding** with a crossed trust boundary is the outcome the mentor is asking for. **"No vulnerability demonstrated within this limited scope"** is equally valid, provided it states what was tested, how often, and what remains untested. What I will not do is present a blocked-attack transcript as though it were a breach, or claim the chatbot is secure on the basis of a small test set.

## 11. Risks and Limitations

- **Permission may not be granted, and that stalls the project.** A public website is not authorization. If no programme scope or written permission is available, active testing will not proceed and I will have no target. I would rather raise this with my mentor before Day 1 than discover it on Day 1.
- **Safe harbour is narrower than it sounds.** It covers only assets named in the programme brief, and it binds the programme owner rather than their infrastructure providers. Most rejected requests will be scope questions, not legal ones.
- **A valid finding may still be closed as informational.** Programmes consistently treat prompt injection without a crossed trust boundary as a known limitation. That is a real possibility even for legitimate work, and it is a reason to aim tests at outcomes rather than at surprising text.
- **The target may have nothing to find.** A public FAQ chatbot has no other users' data, no tools, and nothing privileged, so no amount of cleverness produces a finding. This is a target-selection problem, not a testing problem, and it is the main reason Section 2 constrains what I will pick.
- **Limited visibility.** I may not know the chatbot's model, retrieval sources, or guardrail configuration; findings will describe observed behavior, not speculate about implementation. A result that looks like a bypass may be an undocumented safety classifier.
- **Limited coverage.** One chatbot and a small prompt-injection test set are not a full security assessment, and a single result is not a trend.
- **Indirect injection may not be testable.** It depends on an owner-approved test source; if unavailable, the test will be marked not run.
- **No knowledge-base poisoning.** The Week 8 training-data poisoning lab does not authorize changes to a company's data or model.

- **Responsible disclosure.** Findings remain private and are reported through the owner's designated contact.

## 12. Conclusion

I propose a one-week, permission-based assessment of one existing company website chatbot selected from a published bug bounty or disclosure programme with safe-harbour coverage, focusing on prompt injection techniques covered in the bootcamp. I will not build a separate RAG agent, modify the company's data, or test outside the programme's written scope. The final report will document the methods and tasks actually executed, the responses observed, any reproducible vulnerabilities found, unsuccessful tests, and practical recommendations. No active testing will begin until the chatbot's owner has authorised it in writing or the published programme scope covers the asset.

**Approval requested:** mentor agreement on the target-selection criteria and this prompt-injection scope, plus two specific judgements. First, that selecting only from programmes with safe-harbour coverage is the right gate even though it narrows the choice sharply. Second, that holding findings to a crossed-trust-boundary bar will produce a smaller, more credible report than accepting "the chatbot said something it shouldn't have" as a vulnerability — I expect this to mean reporting no confirmed vulnerabilities is a realistic outcome, and I would rather agree that now than be judged against a result I did not set out to produce. The selected chatbot and final test plan will be recorded only after the system owner has approved the work in writing or the programme scope is confirmed.